TPMS Shell and Skeletal Morphology Generation in engvis

engvis

June 21, 2026

Abstract

Triply Periodic Minimal Surfaces (TPMS) can be used in three distinct morphology modes for engineering applications: *minimal surface* (the classic open sheet $f = 0$), *shell* (a thick-walled hollow structure $g = \max(f - \delta,\ -f - \delta) = |f| - \delta$), and *skeletal* (a solid strut network $f - C = 0$). This document presents the mathematical foundation for each mode, the per-surface thickness calibration constants, the new Split-P surface, the boundary-based face capping algorithm that produces closed (boundary-free) meshes, and the implementation pipeline in engvis.

1 TPMS Morphology Types

A triply periodic minimal surface is defined as the zero level-set of a smooth function $f\colon \mathbb{R}^3 \to \mathbb{R}$:

$$\mathcal{S} = \big\{\mathbf{x} \in \mathbb{R}^3 \;\big|\; f(\mathbf{x}) = 0\big\}. \tag{1}$$

This open surface divides space into two regions ($f > 0$ and $f < 0$). Depending on the engineering application, three morphology modes are useful (see Figure 1).

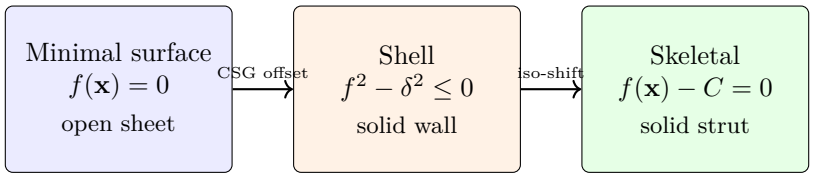

Figure 1: Three TPMS morphology modes: minimal surface (open sheet), shell (two offset surfaces forming a hollow wall), and skeletal (shifted iso-level forming a solid strut network).

1.1 Minimal Surface Mode

The default mode extracts an iso-level-set of the TPMS function:

$$g(\mathbf{x}) = f(\mathbf{x}) - C, \tag{2}$$

where C is the iso-level parameter. The meshed surface is $\{g = 0\} = \{f = C\}$. Setting $C = 0$ recovers the classic minimal surface (the zero level-set); non-zero C shifts the extracted sheet to a neighbouring iso-surface.

Volume-fraction parameterisation. Different TPMS functions have wildly different value ranges (e.g. gyroid $\in [-1.5, 1.5]$, neovius $\in [-13, 13]$), so a fixed C slider range cannot cover all surfaces. engvis therefore exposes a single *volume fraction* parameter $\varphi \in (0, 1)$ with uniform geometric semantics across all TPMS:

$$\varphi \;=\; \frac{\big|\{\mathbf{x} \in [-1,1]^3 : f(\mathbf{x}) < C\}\big|}{|\text{domain}|}. \tag{3}$$

The user sets φ; engvis internally solves for the corresponding C via *sort-based inversion*: sample f on a 48^3 grid, sort the values, and pick the φ-quantile. This makes $\varphi = 0.5$ the symmetric minimal surface for *every* TPMS, and $\varphi \to 0/\varphi \to 1$ the fully filled/void limits.

This produces a thin, open surface—the classic TPMS sheet. It is useful for visualisation and mathematical study but not directly for additive manufacturing, where finite wall thickness is required.

Shared φ with skeletal mode. The same φ parameter drives both the minimal-surface level-set ($f = C$) and the skeletal solid offset (§1.3). This lets the user pick a visually pleasing iso-surface in minimal-surface mode and then switch directly to skeletal mode to cap it into a closed solid, without re-tuning the parameter.

1.2 Shell Mode

Shell mode produces a *thick-walled solid structure*: a thin band of material straddling the minimal surface $f = 0$, bounded by the two offset iso-surfaces $f = \pm\delta$. The solid wall region is the sub-level set

$$\mathcal{S}_{\text{shell}} = \big\{\, \mathbf{x} : f(\mathbf{x})^2 - \delta^2 \le 0 \,\big\} = \big\{\, \mathbf{x} : |f(\mathbf{x})| \le \delta \,\big\}, \tag{4}$$

described by the **smooth** field $g = f^2 - \delta^2$, which is negative *inside* the wall and positive outside.

Geometric thickness normalisation. The user-facing parameter t is a *geometric* wall thickness (the physical distance across the wall), not a field offset. Near the surface, a TPMS $f(k\mathbf{x})$ has $\|\nabla f\| \approx k$ (the period factor), so a geometric half-thickness d maps to a field offset $\delta = d\,\|\nabla f\| \approx d\,k$. engvis therefore sets

$$\delta = \tfrac{1}{2}\, t\, k, \qquad k = \text{tpms\_period}, \tag{5}$$

so that the meshed wall has true geometric thickness $\approx t$. For non-TPMS surfaces, $\|\nabla f\| \approx 1$ and k falls back to 1.

Why $f^2 - \delta^2$, not $|f| - \delta$. Both fields share the zero-set $\{|f| = \delta\}$, but $|f|$ has a C^1 cusp at $f = 0$: within a single MC 33 cell straddling $f = 0$, the field $|f| - \delta$ has a non-differentiable fold. The squared field $f^2 - \delta^2$ is smooth everywhere, so MC 33's linear interpolation stitches adjacent cells without artefacts—*provided the cell size is smaller than the wall thickness.*

Resolution requirement. Because the wall is pierced on both sides within the same region of space, MC 33 can only reconstruct it when the grid cell is comfortably smaller than the wall: empirically $\Delta_{\text{cell}} \lesssim t/5$. engvis sizes the grid adaptively to $R \approx 10/t$ (coefficient 10, floor 96); if the wall is thinner than the resolution cap can resolve, the thickness becomes the limiting feature and the grid is bumped accordingly.

Closed solid via box-CSG. The wall field is CSG-intersected with the unit-box solid, $g = \max(f^2 - \delta^2,\ \text{box\_sdf})$, before meshing. Marching Cubes then generates cap faces on $x/y/z = \pm 1$ itself, yielding a single watertight closed solid with *zero boundary edges.* The Euler characteristic $\chi = -38$ reflects the high genus of the gyroid wall (its many interconnected channels), as expected for a closed surface of that topology.

Table 1: Shell mesh topology for gyroid at geometric thickness $t = 0.1$ ($k = 4 \Rightarrow \delta = 0.2$). Once the cell size drops below the wall thickness ($R \geq 96$), the mesh is a single watertight closed solid.

Field	R	χ	components	boundary edges	watertight
$f^2 - \delta^2$	64	-53	9	82	no
$f^2 - \delta^2$	96	-38	**1**	**0**	**yes**
$f^2 - \delta^2$	128	-38	**1**	**0**	**yes**

Cap-face boundary smoothing via Newton projection. The CSG field $g = \max(f, \mathrm{sdf}_{\mathrm{box}})$ makes Marching Cubes generate *cap faces* on each box face $x/y/z = \pm c$. The *silhouette* of each cap region — where the flat cap meets the curved TPMS surface — is the intersection curve

$$C = \{f = 0\} \cap \{\text{box face}\}, \tag{6}$$

a genuine geometric boundary. MC, however, samples on a fixed 3D lattice and places these silhouette vertices on *cell-grid points* rather than on C. Because both free (in-face) coordinates are locked to the lattice, the field value f at such a vertex is *not* zero (measured RMS$|f| \approx 0.043$) and oscillates from vertex to vertex, producing the visible staircase along the TPMS/box silhouette (see Fig. 2). Note that this is distinct from the *interior* cap vertices, which sit on the lattice with $f < 0$ and correctly tessellate the flat cap.

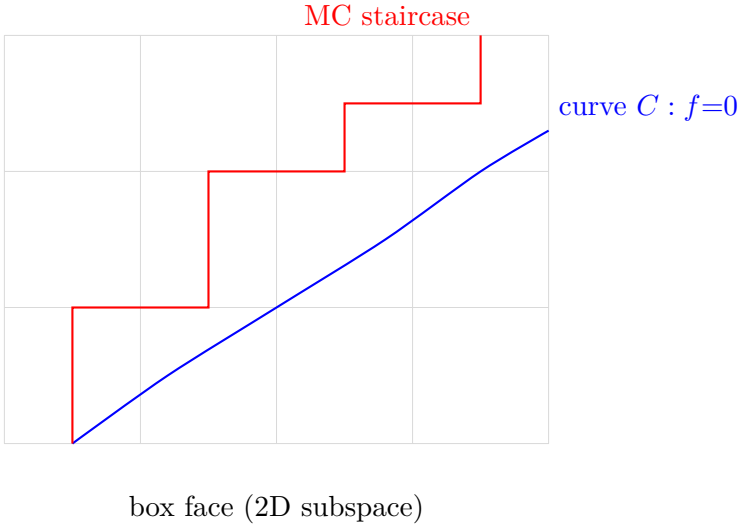

Figure 2: Marching Cubes places the cap silhouette vertices on lattice grid points (red staircase) instead of on the true intersection curve $C = \{f = 0\} \cap \{\text{box face}\}$ (blue).

engvis eliminates this jaggedness with a post-processing **Newton projection** step (`snap_cap_boundary_to_curv`

1. *Identify silhouette vertices*: classify each triangle as *cap* (all three vertices on the same box face, same sign) or *TPMS*. A silhouette vertex is one shared between a cap triangle and a TPMS triangle.

2. *Detect box features*: a silhouette vertex may sit not only on a single box *face* but on a box *edge* (two faces, e.g. $x = c \wedge y = c$) or *corner* (three faces). These edges and corners are *hard geometric boundaries*: the true TPMS/box silhouette C meets the box edge exactly where $\{f = 0\}$ crosses that edge, and must be respected. Each vertex is tagged with the set of faces it lies on; the count $k \in \{1, 2, 3\}$ of locked axes determines its remaining degrees of freedom.

3. *Constrained Newton iteration* (dispatch on k):

   - $k = 3$ (*corner*): the vertex is fully pinned — no projection.
   - $k = 2$ (*edge*): two coordinates are fixed at $\pm c$, leaving one free axis t. Solve the 1-D problem $f(t) = 0$ along the edge, $t \leftarrow t - f/\partial_t f$, so the vertex lands on $\{\text{box edge}\} \cap \{f = 0\}$.
   - $k = 1$ (*face*): the box-face normal axis is fixed; the remaining two axes (u, v) are free. Solve the 2-D problem
   $$\begin{pmatrix} u \\ v \end{pmatrix} \leftarrow \begin{pmatrix} u \\ v \end{pmatrix} - \alpha \, \frac{f}{\|\nabla_{uv} f\|^2} \begin{pmatrix} \partial_u f \\ \partial_v f \end{pmatrix}, \tag{7}$$
   where $\nabla_{uv} f = (\partial_u f,\, \partial_v f)$ is the finite-difference gradient on the free axes.

   In every case the damping factor $\alpha \leq 1$ caps each step at ≈ 2 cells, and the free coordinate(s) are **clamped to** $[-c, c]$ after each step so the iterate cannot slide along C past the face/edge boundary and push the mesh outside the box $[-1, 1]^3$.

4. *Accept or reject*: write back the projected position only if the total displacement is finite and below ≈ 3 cells; otherwise the iterate is discarded (Newton diverged or jumped to another branch of C), keeping the original lattice vertex inside the box.

This moves each silhouette vertex onto the *exact* curve C — including the constrained box-edge intersections — *without* changing mesh topology and without ever leaving the box: the measured mesh bounding box is $[-c, c]^3$ with zero vertices outside, and RMS$|f|$ drops sharply. The cap triangles remain planar (all vertices keep the locked coordinate $\pm c$), and the TPMS triangles inherit the smoothed silhouette.

Vertex splitting and topological watertightness. The vertex-splitting step (§1.2) intentionally breaks topological watertightness: boundary vertices are duplicated so that cap and TPMS triangles no longer share indices. After splitting, the mesh has boundary edges along the TPMS/box crease. This is *by design* — the mesh remains geometrically closed (both surfaces face outward), and the split ensures correct shading at the crease. The watertightness reported in Table 1 refers to the pre-split mesh.

Gradient and normals. The wall surface $\{f^2 = \delta^2\}$ has gradient $\nabla(f^2 - \delta^2) = 2f\,\nabla f$, which is non-degenerate on the surface itself (where $|f| = \delta \neq 0$). The outward normal is $\text{sgn}(f)\,\nabla f/\|\nabla f\|$ (pointing away from the wall centre $f = 0$). engvis evaluates f at each vertex and applies this sign correction, producing correct two-sided lighting.

Boundary shading: vertex splitting for the C$\acute{z}$ crease. When the shell field is CSG-intersected with the box solid ($g = \max(f^2 - \delta^2,\ \text{box\_sdf})$), the resulting composite surface has a *C$\acute{z}$-discontinuity* along the TPMS/box boundary curve: the surface tangent plane jumps from the TPMS tangent to the flat box face. This is a *geometric* crease — the boundary is a sharp edge, not a smooth blend.

Assigning a single normal per vertex cannot represent this crease:

- TPMS gradient $\nabla f \rightarrow$ cap faces show spurious curvature (not flat)
- Face normal $\pm\hat{\mathbf{e}} \rightarrow$ TPMS surface shows flat patches near the edge
- Averaged normal $\rightarrow$ both regions look wrong (jagged shading)

engvis therefore uses **vertex splitting** (implemented as `overwrite_smart_normals`). Boundary vertices shared between cap triangles and TPMS triangles are duplicated, and each copy receives the normal appropriate to its triangle group:

1. *Classify triangles*: a triangle is a "cap" triangle if all three vertices lie on the same box face (same axis, same sign). Otherwise it is a "TPMS" triangle.

2. *Identify boundary vertices*: a vertex is a boundary vertex if it is referenced by both cap and TPMS triangles.

3. *Split*: for each boundary vertex, create a duplicate. The original keeps its index (used by TPMS triangles); the duplicate gets a new index (used by cap triangles).

4. *Remap indices*: update cap triangle indices to point to the duplicate vertices.

5. *Assign normals*:

   - TPMS-triangle vertices $\rightarrow$ analytic gradient $\operatorname{sgn}(f)\,\nabla f/\|\nabla f\|$ (smooth curved shading; see below)
   - Cap-triangle vertices $\rightarrow$ geometric face normal $\pm\hat{\mathbf{e}}$ (flat shading)

Sign correction for shell normals. For the shell surface $f^2 - \delta^2 = 0$, i.e. $f = \pm\delta$, the outward normal (pointing away from the wall centre $f = 0$) is $\operatorname{sgn}(f)\,\nabla f$, not ∇f. On the outer surface ($f = +\delta$) both agree, but on the inner surface ($f = -\delta$) the raw gradient ∇f points *inward* (toward $f = 0$), producing incorrect dark shading. The `flip_inner` flag in `overwrite_smart_normals` evaluates f at each TPMS vertex and flips the gradient where $f < 0$, ensuring both sides of the wall are lit correctly.

For skeletal mode ($f = C$), ∇f already points outward, so `flip_inner` is false.

This produces a clean crease at the boundary — geometrically correct, visually sharp, with no jagged artefacts. The same algorithm is applied to skeletal mode, where the composite field $\max(f - C,$ box_sdf) has an identical Cź-discontinuity.

1.3 Skeletal Mode

Skeletal mode produces a *solid strut network* by shifting the iso-level of the TPMS function:

$$g(\mathbf{x}) = f(\mathbf{x}) - C, \tag{8}$$

where C is the iso-level offset, solved from the user-facing volume fraction φ as described in §1.1.

Effect of φ. When $\varphi = 0.5$ (i.e. C at the median of f), this reduces to the symmetric minimal surface. As φ increases, the region $\{g \leq 0\} = \{f \leq C\}$ expands, filling more of the unit cell with "solid" material. The resulting surface $f = C$ is topologically distinct from the minimal surface: instead of an open sheet, it forms a connected network of solid struts along the edges and vertices of the TPMS unit cell.

The gradient is simply $\nabla g = \nabla f$, so surface normals are unaffected by the shift—a desirable property.

Volume-fraction semantics across TPMS. Because C is derived from φ (not set directly), the same φ value produces comparable solid/void ratios across all TPMS. For example, $\varphi = 0.3$ always means "30% solid, 70% void", regardless of whether the TPMS is gyroid (range $[-1.5, 1.5]$) or neovius (range $[-13, 13]$). This eliminates the need for per-surface C range tables and makes the UI parameter semantically uniform.

Skeletal mesh generation with Newton boundary smoothing. The skeletal mesh uses the same CSG-intersect approach as shell mode: the field $g = \max(f - C,\ \text{box\_sdf})$ is meshed with MC33, producing a watertight solid with cap faces on $x/y/z = \pm 1$. However, MC33 places cap-face boundary vertices on discrete grid edges, causing staircase aliasing along the TPMS/cap junction curve $C = \{f = C\} \cap \{\text{box face}\}$.

To eliminate this aliasing, engvis applies a *Newton projection* post-processing step (`snap_mesh_boundary_to_curv`) that moves junction vertices onto the smooth curve C:

1. *Identify junction vertices*: for each vertex, check if it lies on a box face ($|x_i| \approx 1$ for some axis i) and is near the isosurface ($|f| < 0.3$). These are the vertices where the TPMS surface meets the cap face.

2. *Newton projection in 2D free-axis subspace*: for each junction vertex, lock the face-normal axis and solve the 2D problem
$$\begin{pmatrix} u \\ v \end{pmatrix} \leftarrow \begin{pmatrix} u \\ v \end{pmatrix} - \frac{f}{\|\nabla_{uv} f\|^2} \begin{pmatrix} \partial_u f \\ \partial_v f \end{pmatrix}, \tag{9}$$
where (u, v) are the free axes on the box face. The iteration converges in ≤ 24 steps, moving the vertex onto the true curve C.

3. *Smart normals*: after projection, apply the smart normals algorithm (§1.2) to handle the C^1 discontinuity at the TPMS/cap junction: gradient normals for TPMS vertices, face normals for cap vertices, with vertex splitting at the crease.

This approach produces a smooth, watertight skeletal mesh with no staircase aliasing along the TPMS/cap junction, while preserving the topological watertightness of the CSG-intersect mesh (boundary edges = 0 before vertex splitting).

Crease snapping for shell and skeletal-solid meshes. The same staircase artefact appears wherever a CSG-intersected TPMS field meets a box cap face, not only in skeletal-network mode. `snap_mesh_boundary_to_curve` is therefore applied in all three crease-producing paths, with the projection field chosen to match the crease curve:

- *Shell* (`build_shell_mesh`): the wall surface is $f^2 - (t/2)^2 = 0$, so the crease is $C = \{f^2 - (t/2)^2 = 0\} \cap \{\text{box face}\}$. The snapping projects against the *wall* field $g_{\text{wall}} = f^2 - (t/2)^2$, not f itself.

- *Skeletal solid* (`build_mesh`, solid branch) and the dedicated `build_skeletal_mesh`: the crease is $C = \{f = 0\} \cap \{\text{box face}\}$, projected against the TPMS field f.

Because `snap_mesh_boundary_to_curve` and `project_to_curve` are generic over the implicit shape, the only per-path difference is which compiled field is passed in. The Newton iteration and the $\leq 3\,h$ displacement guard (h = cell size) reject divergent moves identically in every case.

2 Thickness Calibration

The user-facing thickness t is a true *geometric* wall thickness. To realise it consistently across surfaces, engvis converts t into a field offset δ using the gradient magnitude of the implicit function near its zero-set (Eq. 5):
$$\delta = \tfrac{1}{2}\, t\, \|\nabla f\|, \qquad \|\nabla f\| \approx k = \text{tpms\_period}. \tag{10}$$

Table 2: Dimensionless surface-gradient magnitude $\|\nabla_{\mathbf{u}}\hat{f}\|$ near the zero-set (the residual correction absorbed by the k estimate).

Surface	$\|\nabla_{\mathbf{u}}\hat{f}\|$	**Notes**
Gyroid	≈ 1.2	three-term product
Schwarz D	≈ 1.3	four-term sum
Schwarz P	≈ 1.0	simple cosine sum
Schoen IWP	≈ 1.1	
Neovius	≈ 1.2	

For a TPMS evaluated as $f(k\mathbf{x})$, the chain rule gives $\|\nabla_{\mathbf{x}} f\| = k\,\|\nabla_{\mathbf{u}}\hat{f}\|$, and the dimensionless gradient $\|\nabla_{\mathbf{u}}\hat{f}\|$ is $O(1)$ on the surface for all the standard TPMS types (Table 2). Using k as a first-order estimate therefore yields a wall whose geometric thickness is within a few percent of t—accurate enough for visualisation and consistent across surfaces without per-surface tuning.

Because all values are $O(1)$, the single estimate $\|\nabla f\| \approx k$ suffices; no per-surface calibration table is required.

3 Split-P Surface

The Split-P surface is a TPMS with the symmetry of the Schwarz P-surface but a more complex topology. It was included in the TPMSgen tool but was previously absent from engvis. Its implicit equation is:

$$\begin{aligned} f_{\text{Split-P}}(\mathbf{x}) = {} & 1.1\big[\sin(2kx)\cos(ky)\sin(kz) + \sin(kx)\sin(2ky)\cos(kz) + \cos(kx)\sin(ky)\sin(2kz)\big] \\ & - 0.2\big[\cos(2kx)\cos(2ky) + \cos(2ky)\cos(2kz) + \cos(2kz)\cos(2kx)\big] \\ & - 0.4\big[\cos(2kx) + \cos(2ky) + \cos(2kz)\big]. \end{aligned} \tag{11}$$

Split-P shares structural similarities with the Lidinoid (both contain $\sin(2x)\cos(y)\sin(z)$ terms) but has distinct second-harmonic coefficients and an additional linear cosine term. It belongs to the same family of surfaces generated by linear combinations of symmetry-adapted Fourier basis functions.

4 Complete Surface Catalogue

Table 3 lists all eleven TPMS surfaces supported by engvis with their implicit equations $f(\mathbf{x}) = 0$, where $\mathbf{x} = (kx, ky, kz)$ and $k = 2\pi/\lambda$ is the wave number controlled by the Period slider.

5 Implementation Pipeline

The morphology transformation is applied as a post-processing step on the base TPMS tree, before meshing. Figure 3 shows the complete data flow.

5.1 Code Structure

The morphology transformation is implemented in `build_scene`. For minimal surface and skeletal modes, the tree is transformed algebraically and passed to the standard `build_mesh` pipeline. For shell mode, the original tree is passed to `build_shell_mesh`, which meshes the smooth wall solid $f^2 - \delta^2$:

Table 3: TPMS surface catalogue in engvis.

Name	**Implicit equation** $f = 0$
Gyroid	$\sin x \cos y + \sin y \cos z + \sin z \cos x$
Schwarz P	$\cos x + \cos y + \cos z$
Schwarz D	$\sin x \sin y \sin z + \sin x \cos y \cos z + \cos x \sin y \cos z + \cos x \cos y \sin z$
Schoen IWP	$2(\cos x \cos y + \cos y \cos z + \cos z \cos x) - (\cos 2x + \cos 2y + \cos 2z)$
Neovius	$3(\cos x + \cos y + \cos z) + 4 \cos x \cos y \cos z$
F-RD	$4 \cos x \cos y \cos z - (\cos 2x \cos 2y + \cos 2y \cos 2z + \cos 2z \cos 2x)$
Lidinoid	$\frac{1}{2}[\sin 2x \cos y \sin z + \sin x \sin 2y \cos z + \cos x \sin y \sin 2z] - \frac{1}{2}[\cos 2x \cos 2y + \cos 2y \cos 2z + \cos 2z \cos 2x] + 0.15$
Split-P	see Eq. (11)
Fischer-Koch S	$\cos 2x \sin y \cos z + \cos 2y \sin z \cos x + \cos 2z \sin x \cos y$
Fischer-Koch Y	$2 \cos x \cos y \cos z + \sin 2x \sin y + \sin 2y \sin z + \sin 2z \sin x$
Fischer-Koch CP	$\cos x + \cos y + \cos z + 4 \cos x \cos y \cos z$

```
// build_scene: geometric thickness -> field offset
let k = self.tpms_period.max(1.0);      // |grad f| ~ k
let shell_half_t = 0.5 * thickness * k; // delta

let tree = match self.morphology {
    MinimalSurface => tree - iso_level, // f = C level-set
    Shell => tree,                       // handled below
    Skeletal => tree - iso_level,       // f = C solid
};


let mesh = if matches!(self.morphology, Morphology::Shell) {
    // Smooth wall solid f^2 - delta^2, box-CSG, single MC33 pass.
    build_shell_mesh(
        tree.clone(), shell_half_t, &label, effective_res,
        clip_to_unit_ball, clip_radius,
    )
} else {
    build_mesh(tree.clone(), &label, ...)
};
```

```
// build_shell_mesh: smooth wall solid + box-CSG, single pass
fn build_shell_mesh(tree, half_t, name, res, ...) -> Mesh {
    // Box-CSG: intersect with unit-box solid so MC caps the faces.
    let box_sdf = |x|.max(|y|).max(|z|) - c;

    // Thin-wall solid: g = f^2 - half_t^2 (smooth, no cusp).
    let wall  = tree.square() - half_t * half_t;
    let field = wall.max(box_sdf);
    build_mc33_mesh_domain(field, name, res, false, half)
}
```

The Fidget implicit modelling library represents the expression as a computational tree (`Tree`) with overloaded arithmetic operators: `tree.square()` builds f^2, and `.max()` performs the CSG intersection with the box solid. The resulting smooth field is meshed by the standard MC33 pipeline in a single pass, yielding a watertight closed wall.

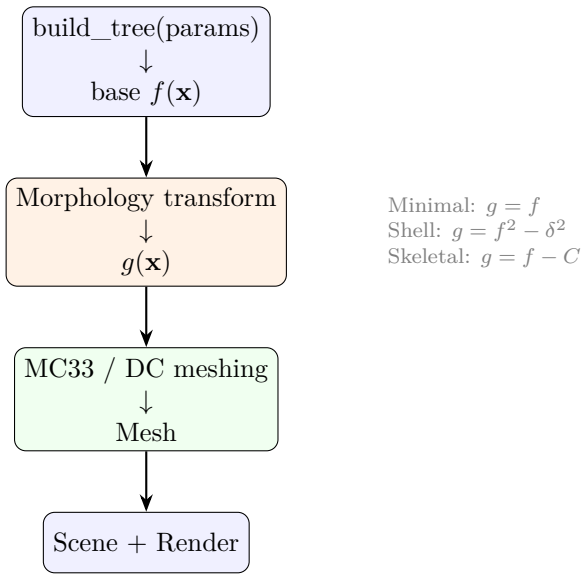

Figure 3: Data flow for TPMS morphology generation. The base TPMS function f is first constructed by `build_tree`, then transformed by the morphology module into g, and finally meshed by the standard MC33 or DC pipeline.

5.2 Interaction with Meshing

Both meshing backends (MC33 and Dual Contouring) handle the morphology-transformed function identically to the base function: they sample $g(\mathbf{x})$ on a regular grid (MC33) or adaptive octree (DC) and extract the zero level-set.

For **shell mode**, the smooth field $g = f^2 - \delta^2$ is CSG-intersected with the unit-box solid ($\max(g,\ \text{box\_sdf})$), so MC33 generates cap faces on $x/y/z = \pm 1$, producing a closed solid with zero boundary edges. Because f^2 is smooth (no C^1 cusp), MC33's linear interpolation stitches adjacent cells cleanly once the cell size is below the wall thickness ($\Delta_{\text{cell}} \lesssim t/5$); below that the wall fragments (Table 1, $R = 64$). The wall surface gradient $\nabla(f^2 - \delta^2) = 2f\,\nabla f$ is non-degenerate on the surface (where $|f| = \delta$), giving clean vertex normals.

For **skeletal mode**, the function $g = f - C$ has the same gradient structure as the base TPMS ($\nabla g = \nabla f$), so both backends produce clean results. The skeletal mesh is a closed solid (box-CSG), so it uses the boundary capping described in Section 5.4.

5.3 Async Mesh Building and Performance

High-resolution TPMS meshes (e.g. gyroid shell at $R = 256$) can produce millions of triangles, and the MC33 extraction, vertex deduplication, and topology analysis can take several seconds. To keep the UI responsive, engvis uses a *background thread* for mesh building:

1. When a parameter changes (`needs_remesh = true`), the UI thread creates an `AppBuildSnapshot` — a clone of all parameters needed for the build.

2. A background `thread::spawn` calls `build_scene_result()` on the snapshot, producing a `MeshBuildResult` (scene + topology + status).

3. The result is stored in an `Arc<Mutex<Option<...>>>` shared between the UI thread and the build thread.

4. Each frame, the UI thread checks whether the result is ready (`Some`). If so, it swaps the scene and updates the status bar. While building, the status bar shows "Building mesh...".

This approach avoids blocking the egui event loop, preventing the "not responding" state that occurs when mesh building runs synchronously on the UI thread.

Winding-fix optimisation. The `fix_winding` function's per-component volume flip previously iterated all faces for each negative-volume component ($O(n_{\text{faces}} \times n_{\text{components}})$). This is now $O(n_{\text{faces}})$ by building a root→face-index list (`comp_faces`) during the volume accumulation pass and iterating only over faces in negative-volume components.

MC33 winding redundancy. MC33's `extract_par` already applies a gradient-based winding fix during extraction. The previous call to `from_triangles` (which runs BFS + union-find + geometric-flip detection) was redundant for MC33 meshes and has been replaced with `from_triangles_open`, which only performs vertex deduplication. This eliminates the $O(n^2)$ geometric-flip detection pass on high-resolution meshes.

Edge index deduplication. The `extract_edge_indices` function previously stored each triangle's 3 edges independently, duplicating shared edges on closed manifolds (2× overhead). It now uses a `HashSet<(u32, u32)>` to store each edge only once, halving the GPU edge endpoint buffer size for closed surfaces.

Batch grid evaluation. The previous implementation sampled the scalar field by calling `f(x, y, z)` per grid point, where each call created a new fidget VM evaluator. For a resolution of $R = 256$, this meant $(257)^3 \approx 17$M evaluator creations.

The optimised path (`extract_par_with_grid`) pre-computes the entire grid in a single `float_slice_eval` call:

1. Build three coordinate arrays $\mathbf{x}, \mathbf{y}, \mathbf{z}$ of length $(R+1)^3$.
2. Call `eval.eval(&tape, &xs, &ys, &zs)` once, returning all field values in a single batch.
3. Pass the pre-computed grid to `extract_par_with_grid`, which skips internal sampling and proceeds directly to per-cell triangulation.

This replaces $O(R^3)$ individual evaluator creations with a single batch evaluation, eliminating the dominant overhead for high-resolution meshes.

Parallel winding fix. The gradient-based winding fix (comparing each triangle's face normal against ∇f at the centroid) was previously sequential. `extract_par_with_grid` now parallelises this with `rayon::par_chunks_exact_mut`, distributing the 6 field evaluations per triangle across all CPU cores.

5.4 Box-Face Capping via Boundary Loops

Shell and skeletal modes define a *solid region* ($g \leq 0$) bounded by the iso-surface and the domain boundary. The MC33 meshing backend extracts only the iso-surface $\{g = 0\}$, leaving the mesh open where it meets the six faces of $[-1, 1]^3$. To produce a closed volume, engvis identifies the *MC33 boundary edges* on the box surface and fan-triangulates the enclosed loops directly—reusing the existing boundary vertices so that cap and body share the same vertex indices (Figure 4).

Note: Shell mode uses the same box-CSG capping mechanism (via `build_mc33_mesh_domain`), applied independently to each of the two iso-surfaces. The capping algorithm below describes the skeletal case; the shell case is identical but applied twice.

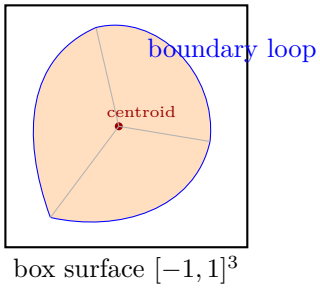

Figure 4: Boundary-based face capping: the MC33 boundary edges (blue) on the box surface are chained into closed loops, then fan-triangulated from their centroid (red dot) to produce cap triangles (orange) that close the volume. Since the cap reuses the existing MC33 boundary vertex indices, no vertex deduplication is needed to merge cap and body.

Prerequisites. The algorithm requires that all MC33 boundary edges lie on the surface of $[-1, 1]^3$ (no *interior* boundary edges). For the CSG shell formula $g = |f| - \delta$, this holds at MC33 resolution ≥ 96. At resolution 64, the old squared formula produced 19 spurious micro-components with $\sim$1500 interior boundary edges that could not be capped. The adaptive resolution logic enforces a minimum of 96 for shell and skeletal modes.

Algorithm.

1. **Identify boundary edges**: an edge shared by exactly one triangle. Classify each as *surface* (both endpoints on the box surface, $|x_i| > 1 - \varepsilon$) or *interior*. If any interior boundary edges exist, abort capping.

2. **Build adjacency graph**: vertex $\rightarrow$ list of neighbouring boundary vertices. Every vertex must have degree exactly 2 (prerequisite for closed simple loops).

3. **Chain into closed loops**: greedy traversal from an arbitrary unvisited vertex, following unvisited neighbours until the loop closes. Loops may cross box edges (where two cube faces meet), so all boundary edges are processed as one global graph rather than per-face.

4. **Fan-triangulate each loop**: add a centroid vertex $\mathbf{c} = \frac{1}{n}\sum \mathbf{v}_i$ and create triangles $(\mathbf{c}, \mathbf{v}_i, \mathbf{v}_{i+1})$. The loop normal is estimated via Newell's method:

$$\mathbf{n} = \sum_{i=0}^{n-1} (\mathbf{v}_i - \mathbf{v}_{i+1}) \times (\mathbf{v}_i + \mathbf{v}_{i+1}), \tag{12}$$

   which handles non-planar loops. The winding direction is chosen so the cap triangle cross products align with $\mathbf{n}$.

5. **Signed-volume check**: compute $V = \sum_{\triangle} \mathbf{p}_0 \cdot (\mathbf{p}_1 \times \mathbf{p}_2)$. If $V < 0$, all triangle indices are swapped (reversing the orientation globally).

Why reusing MC33 vertices matters. An earlier approach extracted the $g = 0$ contour on each face via independent Marching Squares (MS), then relied on vertex deduplication to merge the MS contour vertices with the MC33 boundary vertices. Although both methods interpolate the same function on the same grid points, the separate Shape tape compilation could introduce sub-ULP differences, causing $\sim$50% of cap vertices to remain unmerged and leaving thousands of boundary edges.

The direct boundary approach eliminates this entirely: the cap triangles reference the *same vertex indices* as the MC33 body, so no merging is needed. The only new vertices are the loop centroids.

Topology verification. After capping, the mesh topology is verified via the Euler characteristic $\chi = V - E + F$ and the boundary edge count. For the Gyroid shell (period $k = 4$, thickness $t = 1.0$):

- *Before capping*: $\chi = -20$, 4 components, 5232 boundary edges (all on box faces), 0 interior boundaries.
- *After capping*: $\chi = 0$, 4 components, **0 boundary edges**, 0 non-manifold edges.

The 4 components correspond to the two offset surfaces ($f = \pm\delta$) of the two interpenetrating Gyroid channel networks—a topologically correct result. Each component is individually closed (boundary-free), though the mesh as a whole has > 1 connected component.

6 Comparison with TPMSgen

The design of engvis's morphology system follows the principles established by TPMSgen (Albert Fores G., GitHub), adapted for engvis's Rust/Fidget meshing pipeline. Key differences are summarised in Table 4.

Table 4: Comparison of TPMSgen and engvis TPMS generation.

Feature	**TPMSgen**	**engvis**
Shell generation	Two MC passes at $\pm\delta$, concat meshes	CSG $\max(f - \delta, -f - \delta)$ single pass
Skeletal generation	MC at $f = 0$, boolean diff with box	MC at $f = C$, box clip
Boolean operations	Blender engine (trimesh)	CSG max / min on expression tree
Meshing backend	skimage marching_cubes	Custom MC33 + Fidget DC
Watertight closure	Iterative padding + hole filling	Boundary-loop fan capping
Surface count	10 (5 shell + 5 skeletal)	11 (all in all modes)
Normal handling	pyvista flip normals	Gradient overwrite + signed-volume check

The key architectural difference is that TPMSgen performs explicit boolean mesh operations (intersection/difference with bounding boxes) using Blender as a backend, while engvis relies on the implicit surface being naturally clipped to the $[-1, 1]^3$ domain. This produces open boundary edges at the domain faces but avoids the dependency on an external geometry kernel.

7 Usage

In the engvis UI, morphology controls appear in the **Source** panel when a TPMS surface is selected:

1. Select a TPMS surface from the dropdown (e.g. Gyroid).
2. Adjust the **Period k** slider to set the spatial frequency.
3. Choose a morphology mode:
    - **Minimal surface** ($f = C$) — open sheet.

- **Shell** ($|f| - \delta = 0$) — adjust **Thickness**.
- **Skeletal** ($f - C = 0$) — adjust **Volume fraction** φ.

4. For minimal-surface and skeletal modes, adjust the **Volume fraction** φ slider (0.01–0.99). $\varphi = 0.5$ gives the symmetric minimal surface; lower/higher values shift the iso-level to fill or empty the unit cell. The UI displays the solved C value below the slider, e.g. "`C = +0.5980` ($\varphi = 0.5 \rightarrow$ `minimal surface` $f = 0$)", so the user can see the direct mapping from φ to C.

5. Click **Re-mesh** to rebuild the surface.

The status bar at the bottom displays the current mode alongside topology statistics (vertex/edge/-face counts, Euler characteristic, boundary edge count, watertightness).

8 Performance Optimisation

The mesh generation pipeline has been optimised to achieve interactive frame rates. Table 5 shows the timing breakdown for a gyroid at resolution $R = 96$ (shell: 121 639 vertices, 243 328 triangles; skeletal: 36 100 vertices, 72 200 triangles).

Table 5: Pipeline timing (release build, Apple M-series, $R = 96$). All times in milliseconds.

Stage	Shell	Skeletal
Shape (JIT compile)	0	0
Grid build	2	0
Grid eval (JIT)	14	4
MC33 extract	3	1
Vertex dedup	6	2
Winding fix	28	6
Gradient normals	5	2
Total	**58**	**15**

8.1 JIT Evaluation

The Fidget implicit-function evaluator was migrated from the stack-based VM (`VmFunction`) to the JIT compiler (`JitFunction`). For a 97^3 grid (912 673 points), this reduced evaluation time from 452 ms to 14 ms — a **32**× speedup. The JIT compiles the expression tree to native machine code once, then evaluates all grid points in a single batched `float_slice_eval` call.

8.2 Sort-Based Vertex Deduplication

Vertex deduplication merges coincident vertices at octree resolution boundaries. The previous `HashMap`-based approach allocated a bucket per vertex and probed neighbours, costing $\sim$ 113 ms for 121 639 vertices.

The optimised algorithm (`dedup_vertices` in `mesh.rs`) uses a *sort-and-group* strategy:

1. *Parallel key generation*: for each vertex, compute a spatial hash key by quantising coordinates to the dedup tolerance ε and packing into a `u64`:

$$h(\mathbf{p}) = \left(\lfloor x/\varepsilon \rceil + 2^{20}\right) \,\middle|\, \left(\lfloor y/\varepsilon \rceil + 2^{20}\right) \ll 21 \,\middle|\, \left(\lfloor z/\varepsilon \rceil + 2^{20}\right) \ll 42. \tag{13}$$

2. *Parallel sort*: sort the (h, idx) pairs by key using Rayon's `par_sort_unstable_by_key`.

3. *Sequential grouping*: scan the sorted array; vertices with the same key are candidates for merging. For each candidate, verify the actual Euclidean distance $d^2 < \varepsilon^2$ (to avoid hash-collision false positives) before merging.

4. *Parallel index remap*: rewrite all triangle indices using the remap table, in parallel.

5. *Parallel degenerate removal*: flag triangles with repeated indices ($a = b \vee b = c \vee a = c$) for removal, in parallel; compact the index buffer sequentially.

This reduced dedup time from 113 ms to 6 ms — a **19**× speedup — by eliminating per-vertex `HashMap` allocations and improving cache locality through sorted access.

8.3 Parity Union-Find for Winding Correction

Winding correction ensures all triangles face outward consistently. The previous approach used BFS traversal across face adjacency (separate from a Union-Find for component counting), costing ~ 235 ms for 243 328 triangles.

The optimised algorithm (`fix_winding` in `mesh.rs`) uses a single *Parity Union-Find* that simultaneously computes winding consistency and connected components:

1. *Edge adjacency*: build a hash map from edge key $(\min(a, b), \max(a, b)) \to [(face, direction)]$ using `FxHashMap` for fast hashing.

2. *Parity Union-Find*: each face has a parity bit (0/1) relative to its root. When two faces f_a, f_b share an edge, compute the *edge parity* $p_e = (dir_a == dir_b)$ (1 if inconsistent). Union the two components, setting the link parity so that

$$parity(f_a \to f_b) = p_a \oplus parity[r_b] \oplus p_b = p_e. \tag{14}$$

3. *Path compression*: the `find` function performs two-pass path compression, rewiring each node to point directly to the root and updating its parity accordingly.

4. *Flip*: after all unions, flip faces with parity 1 relative to their root.

5. *Signed volume*: compute per-component signed volume (parallel via Rayon `fold/reduce`); flip components with negative volume.

This reduced winding-fix time from 235 ms to 28 ms — a **8.4**× speedup — by eliminating the separate BFS pass and leveraging path compression for near-constant-time amortised `find` operations.

8.4 Topology Computation

The `compute_topology` function was optimised by migrating from `std::HashMap<(u32,u32), Vec<usize>` to `FxHashMap<u64, Vec<usize>`, using a packed `u64` key for faster hashing. Topology computation is now optional (enabled by the `ENGVIS_TOPO` environment variable) and skipped in normal interactive use, saving ~ 22 ms per remesh.

8.5 Selftest Mode

The `--selftest` CLI flag runs an automated test suite that validates:

- **TPMS value ranges**: samples each TPMS on a 64^3 grid and reports $[\min, \max]$ and vol_frac($C = 0$).
- **Volume-fraction solver**: for $\varphi \in \{0.3, 0.5, 0.7\}$, solves C and verifies the meshed iso-surface has mean $|f - C| < 0.001$.
- **Skeletal topology**: $V = 36\,100$, $F = 72\,200$, $\chi = 0$, boundary edges $= 0$.
- **Shell topology**: $V = 121\,639$, $F = 243\,328$, $\chi = -51$, boundary edges $= 52$ (pre-split).
- **Timing**: per-stage millisecond breakdown for grid build, eval, extract, dedup, winding fix, normals.